

```

1  #include <stdint.h>
2
3  struct bit_7{
4      unsigned char value; // 7 bit
5  };
6
7  struct bit_3{
8      unsigned char value; // 3 bit
9  };
10
11 struct bit_2{
12     unsigned char value; // 2 bit
13 };
14
15 struct bit_10{
16     int16_t value; // 10 bit
17 };
18
19 struct bit_35{
20     int64_t value; // 35 bit
21 };
22
23 struct Tag{
24     bool operator== (const Tag & other) const{
25         return value.value == other.value.value;
26     }
27     bit_35 value;
28 }; //35 bit
29
30 struct Address{
31     Tag tag;
32     bit_10 index;
33     bit_7 offset;
34 }; // 52 bit
35
36 struct Cache_string{
37     bool valid;
38     Tag saved_tag;
39     unsigned char data;
40     bit_3 life_time; // 3 bit
41     bool modified;
42 };
43
44 struct Inner_state{
45     Cache_string S[8] = {}; //S0 === all zeros
46 };
47
48 enum class Opcode : uint8_t{
49     Noop = 0,
50     Read,
51     Write,

```

```

52     Snoop
53 };
54
55 struct Input_data{
56     Opcode opcode;
57     Tag tag;
58     unsigned char data;
59 };
60
61 struct Output_data{
62     bool valid;
63     Tag tag;
64     unsigned char data;
65 };
66
67 struct Cache{
68     Inner_state states[1024];
69 };
70
71
72 static unsigned char ddr5[1024] = {0};
73 int current_index = 0;
74
75 int64_t merge(Tag tag, int index){
76     return tag.value.value + (index << 35);
77 }
78
79 unsigned char memory(Tag tag){
80     return ddr5[merge(tag, current_index)%1024];
81 }
82
83 void memory(Tag tag, unsigned char new_data){
84     ddr5[merge(tag, current_index)%1024] = new_data;
85 }
86
87 void state_change(Inner_state & state, const Input_data &input){
88
89     switch (input.opcode)
90     {
91         case Opcode::Read:
92             {
93                 int hit_n = -1;
94                 int last_free_n = -1;
95                 for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
96                     auto & string = state.S[i];
97                     if(!string.valid){
98                         last_free_n = i;
99                         continue;
100                     }
101                     if(!(string.saved_tag == input.tag)){
102                         continue;

```

```

103     }
104     hit_n = i;
105     break;
106 }
107
108 if(hit_n != -1){ // if hit
109     for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
110         auto & string = state.S[i];
111         if(i == hit_n){
112             string.life_time.value = 0;
113         }
114         else{
115             string.life_time.value++; // Least recently read counter increase
116         }
117     }
118 }
119 else if(last_free_n != -1){ // if miss, but cache not full
120     auto & free_string = state.S[last_free_n];
121     free_string.data = memory(input.tag);
122     free_string.life_time.value = 0;
123     free_string.saved_tag = input.tag;
124     free_string.modified = false;
125     free_string.valid = true;
126
127     for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
128         auto & string = state.S[i];
129         if(i != last_free_n){
130             string.life_time.value++; // Least recently read counter increase
131         }
132     }
133 }
134 else{// if miss, and have to make free
135     int lrr_n = -1;
136     int lrr_value = -1;
137     for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
138         auto & string = state.S[i];
139         if(lrr_value < string.life_time.value){
140             lrr_n = i;
141             lrr_value = string.life_time.value;
142         }
143     }
144
145     auto & writeback_string = state.S[lrr_n];
146     if(writeback_string.modified){
147         memory(writeback_string.saved_tag, writeback_string.data);
148     }
149     writeback_string.data = memory(input.tag);
150     writeback_string.life_time.value = 0;
151     writeback_string.saved_tag = input.tag;
152     writeback_string.modified = false;
153     writeback_string.valid = true;

```

```

154         for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
155             auto & string = state.S[i];
156             if(i != lrr_n){
157                 string.life_time.value++; // Least recently read counter increase
158             }
159         }
160     }
161 }
162 break;
163 case Opcode::Write:
164 {
165     int hit_n = -1;
166     for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
167         auto & string = state.S[i];
168         if(!string.valid){
169             continue;
170         }
171         if(!(string.saved_tag == input.tag)){
172             continue;
173         }
174         hit_n = i;
175         break;
176     }
177
178     if(hit_n != -1){ // if hit
179         auto & hit_string = state.S[hit_n];
180         hit_string.data = input.data;
181         hit_string.modified = true;
182     }
183     else{ // if miss
184         memory(input.tag, input.data);
185     }
186 }
187 break;
188 case Opcode::Snoop:
189 {
190     for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
191         auto & string = state.S[i];
192         if(!string.valid){
193             continue;
194         }
195         if(!(string.saved_tag == input.tag)){
196             continue;
197         }
198         if(string.modified){ //writeback
199             memory(input.tag, string.data);
200         }
201         string.valid = false; //invalidate
202     }
203     break;
204 default:

```

```

205         break;
206     }
207 }
208
209 Output_data output(const Inner_state & state, const Input_data &input){
210
211     Output_data res = {}; // valid = false
212
213     switch (input.opcode)
214     {
215         case Opcode::Read:
216             for(int i = 0; i < sizeof(state.S)/sizeof(Cache_string); i++){
217                 auto & string = state.S[i];
218                 if(!string.valid){
219                     continue;
220                 }
221                 if(!(string.saved_tag == input.tag)){
222                     continue;
223                 }
224                 res.valid = true;
225                 res.tag = input.tag;
226                 res.data = string.data;
227                 break;
228             }
229             break;
230         case Opcode::Write: //do nothing
231             break;
232         case Opcode::Snoop: //do nothing
233             break;
234         default:
235             break;
236     }
237
238     return res;
239 }
240
241 int test_write_to_mem_minimal(){
242
243     //Note: write to memory occurs with first instruction since "политика заведения - промах по чтению"
244     Input_data input[1] = {
245         {
246             Opcode::Write, //Opcode
247             {{0xBEDA}}, //Tag
248             0xf //Data
249         }
250     };
251     Cache cache = {};
252     const int index = 112;
253     auto & state = cache.states[index];
254     current_index = index;
255     for(int i = 0; i < sizeof(input)/sizeof(Input_data); i++){

```

```

256         auto output_data = output(state, input[i]);
257         state_change(state, input[i]);
258     }
259     /*
260     S0 -> S0 / {0, 0, 0}
261     // Запись по адресу: {tag, index}
262     */
263     return 0;
264 }
265
266 int test_free_from_cache(){
267
268     Input_data input[9] = {
269     {
270         Opcode::Read, //Opcode
271         {{0x0DED}}, //Tag
272         0x0 //Data
273     },
274     {
275         Opcode::Read, //Opcode
276         {{0x1DED}}, //Tag
277         0x0 //Data
278     },
279     {
280         Opcode::Read, //Opcode
281         {{0x2DED}}, //Tag
282         0x0 //Data
283     },
284     {
285         Opcode::Read, //Opcode
286         {{0x3DED}}, //Tag
287         0x0 //Data
288     },
289     {
290         Opcode::Read, //Opcode
291         {{0x4DED}}, //Tag
292         0x0 //Data
293     },
294     {
295         Opcode::Read, //Opcode
296         {{0x5DED}}, //Tag
297         0x0 //Data
298     },
299     {
300         Opcode::Read, //Opcode
301         {{0x6DED}}, //Tag
302         0x0 //Data
303     },
304     {
305         Opcode::Read, //Opcode
306         {{0x7DED}}, //Tag

```

```

307         0x0 //Data
308     },
309     {
310         Opcode::Read, //Opcode
311         {{0xBEDA}}, //Tag
312         0x0 //Data
313     }
314 };
315 //Note: 8 reads for fill of cache and 9th will trigger read with free of one cache string
316 Cache cache = {};
317 const int index = 112;
318 current_index = index;
319 auto & state = cache.states[index];
320 for(int i = 0; i < sizeof(input)/sizeof(Input_data); i++){
321     auto output_data = output(state, input[i]);
322     // do stuff with output
323     state_change(state, input[i]);
324 }
325 /*
326     Sn : {8x{valid, tag, data, life_time, modified}}
327     X : {opcode, tag, data}
328     Y : {valid, tag, data}
329
330     Ss -> Sf: {Sf} / {Y}
331
332     S0 -> S1: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0,
333     0, 0, 0, 0}, {1, 0x0DED, 0x0, 0, 0}} / {1, 0x0DED, 0x0}
334     S1 -> S2: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {1,
335     0x1DED, 0x1, 0, 0}, {1, 0x0DED, 0x0, 1, 0}} / {1, 0x1DED, 0x1}
336     S2 -> S3: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {1, 0x2DED, 0x2, 0,
337     0}, {1, 0x1DED, 0x1, 1, 0}, {1, 0x0DED, 0x0, 2, 0}} / {1, 0x2DED, 0x2}
338     S3 -> S4: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {1, 0x3DED, 0x3, 0, 0}, {1, 0x2DED,
339     0x2, 1, 0}, {1, 0x1DED, 0x1, 2, 0}, {1, 0x0DED, 0x0, 3, 0}} / {1, 0x3DED, 0x3}
340     S4 -> S4: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {1, 0x4DED, 0x4, 0, 0}, {1, 0x3DED, 0x3, 1, 0}, {1,
341     0x2DED, 0x2, 2, 0}, {1, 0x1DED, 0x1, 3, 0}, {1, 0x0DED, 0x0, 4, 0}} / {1, 0x4DED, 0x4}
342     S5 -> S6: {{0, 0, 0, 0, 0}, {0, 0, 0, 0, 0}, {1, 0x5DED, 0x5, 0, 0}, {1, 0x4DED, 0x4, 1, 0}, {1, 0x3DED, 0x3, 2, 0},
343     {1, 0x2DED, 0x2, 3, 0}, {1, 0x1DED, 0x1, 4, 0}, {1, 0x0DED, 0x0, 5, 0}} / {1, 0x5DED, 0x5}
344     S6 -> S7: {{0, 0, 0, 0, 0}, {1, 0x6DED, 0x6, 0, 0}, {1, 0x5DED, 0x5, 1, 0}, {1, 0x4DED, 0x4, 2, 0}, {1, 0x3DED, 0x3,
345     3, 0}, {1, 0x2DED, 0x2, 4, 0}, {1, 0x1DED, 0x1, 5, 0}, {1, 0x0DED, 0x0, 6, 0}} / {1, 0x6DED, 0x6}
346     S7 -> S8: {{1, 0x7DED, 0x7, 0, 0}, {1, 0x6DED, 0x6, 1, 0}, {1, 0x5DED, 0x5, 2, 0}, {1, 0x4DED, 0x4, 3, 0}, {1, 0x3DED,
347     0x3, 4, 0}, {1, 0x2DED, 0x2, 5, 0}, {1, 0x1DED, 0x1, 6, 0}, {1, 0x0DED, 0x0, 7, 0}} / {1, 0x7DED, 0x7}
348     S8 -> S9: {{1, 0x7DED, 0x7, 1, 0}, {1, 0x6DED, 0x6, 2, 0}, {1, 0x5DED, 0x5, 3, 0}, {1, 0x4DED, 0x4, 4, 0}, {1, 0x3DED,
349     0x3, 5, 0}, {1, 0x2DED, 0x2, 6, 0}, {1, 0x1DED, 0x1, 7, 0}, {1, 0xBEDA, 0x8, 0, 0}} / {1, 0xBEDA, 0x8}
350 */
351 return 0;
352 }
353
354 /*
355     Количество состояний : 2^(8*(1+35+8+3+1)) = 2^384
356     Пройдено: 9
357     Процент покрытия КА: 9/2^384 ~ 2.28e-113%

```

```
349
350     Количество переходов : Количество состояний * Размер входных данных = 2^384 * 2^(2 + 35 + 8) = 2^384 * 2^45 = 2^429
351     Пройдено: 10
352     Процент покрытия КА: 10/2^429 ~ 7.21e-127%
353
354     Код на с++:
355     gcov: 73.53%
356 */
357
358
359 int main(){
360
361     test_write_to_mem_minimal();
362     test_free_from_cache();
363     return 0;
364 }
365
366
```